```
clear all;clc;
%å?,æ•°è®¾ç½®
Nt = 256;
f = 60e9;
c = 3e8;
K = 1; % ç”¨æˆ·æ•°é‡?
L = 1; % æ¯?ä¸ªç”¨æˆ·çš„è·¯å¾„æ•°ï¼ˆå…ˆè€ƒè™‘LOSå¾„ï¼‰
lambda = c/f;
d = lambda / 2;
N_iter = 10;
theta_min = - pi / 2;
theta_max = pi / 2;

%é??å□†ç®—æ³•BSç«¯ ?æœ¬
[w_far_BS] = exhaustive_codebook(Nt);
%BSç«¯é‡?å¤?ç ?ç ?æœ¬
[w_hierarchy_far] = hierarchy_codebook(Nt);
%BSç«¯ä°Œå^†æ³•å^†å±‚ ?æœ¬
[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);
%BSç«¯conv codeå^†å±‚ ?æœ¬(adaptive)
setup_encoder;
[Codebook] = hierarchy_conv_codebook(Nt,conv_encoder_conf);

%% Start Simulation
t0 = clock;
SNR_dB_list=[-5:3:10];
SNR_list = 10.^(SNR_dB_list/10.);
SNR_len = length(SNR_dB_list);

rate_far_exhaustive  = zeros(length(SNR_list),N_iter);
rate_tra = zeros(length(SNR_list),N_iter);
rate_softconv_hierarchy = zeros(length(SNR_list),N_iter);
rate_repeat_hierarchy = zeros(length(SNR_list),N_iter);


for idx = 1:SNR_len
    SNR_dB = SNR_dB_list(idx);
    SNR = SNR_list(idx);
    fprintf('SNR_dB = %.4f[%d/%d] | run %.4f s\n', SNR_dB,  idx, SNR_len,
etime(clock, t0));
    for i_iter = 1:N_iter
        for k = 1:K
            theta_DoA(1,k) = unidrnd(Nt);
            H=zeros(Nt,K);
            h_BS=zeros(Nt,K);
            [H(:,k),h_BS(:,k)] =
generate_channel(Nt,theta_DoA(1,k),d,lambda);
        end
        %é??å□†æ?œç´¢ç®—æ³•
        for k = 1:K
            [array_gain_exhaustive(1,k),id_BS_exhaustive(1,k)] =
training_exhaustive(w_far_BS,H(:,k),SNR,Nt);
            rate_far_exhaustive(idx, i_iter) =rate_far_exhaustive(idx,
i_iter)+ log2(1 + SNR * array_gain_exhaustive(1,k));
        end
        %ä¼ ç»Ÿå^†å±‚ ?æœ¬
        for k = 1:K
```

```
            [array_gain_tra(1,k),id_BS_tra(1,k)] =
training_hierarchy_tra(w_hierarchy_tra,H(:,k),SNR,Nt);
            rate_tra(idx, i_iter) =rate_tra(idx, i_iter)+ log2(1 + SNR *
array_gain_tra(1,k));
        end
        %é‡?å¤?ç¼-ç ?åˆ†å±‚è®-ç»ƒ
        [array_gain_repeat,id_BS_repeat] =
training_hierarchy_repeat(w_hierarchy_far,Nt,H,K,SNR);
        for k = 1:K
            rate_repeat_hierarchy(idx, i_iter)
=rate_repeat_hierarchy(idx, i_iter)+ log2(1 + SNR *
array_gain_repeat(1,k));
        end
        %å?·ç§¯ç ?åˆ†å±‚è®-ç»ƒ-è½¯è¯‘ç ?(adpative)
        for k = 1:K
            [array_gain_softconv(1,k),id_BS_softconv(1,k)] =
training_hierarchy_softconv(Codebook,H(:,k),SNR,Nt,conv_encoder_conf);
            rate_softconv_hierarchy(idx, i_iter)
=rate_softconv_hierarchy(idx, i_iter)+ log2(1 + SNR *
array_gain_softconv(1,k));
        end
    end
end

%% plot figure
rate_tra = mean(rate_tra,2);
rate_far_exhaustive= mean(rate_far_exhaustive,2);
rate_softconv_hierarchy= mean(rate_softconv_hierarchy,2);
rate_repeat_hierarchy = mean(rate_repeat_hierarchy,2);

C = linspecer(7);
maker_num = 25;
figure; hold on; box on; grid on;
p1 = plot(SNR_dB_list, rate_softconv_hierarchy, '-+', 'color',C(1, :) ,
'Linewidth', 1.6,'MarkerFaceColor','w');
p2 = plot(SNR_dB_list, rate_tra, '-o', 'color',C(2, :) , 'Linewidth',
1.6,'MarkerFaceColor','w');
p3 = plot(SNR_dB_list, rate_far_exhaustive, '-s','color', C(3, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
p4 = plot(SNR_dB_list, rate_repeat_hierarchy,  '-d','color', C(4, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
xlabel('SNR (dB)', 'interpreter', 'latex', 'fontsize', 12)
ylabel('Average Rate (bit/s/Hz)', 'interpreter', 'latex', 'fontsize', 12)
legend([p1,p2,p3,p4],{'Proposed CBT', 'Traditional hierarchical
BT','Exhaustive BT','Repetitive code-based BT'}, 'interpreter', 'latex',
'fontsize', 12);
```

```
clear all;clc;
%å?,æ•°è®¾ç½®
Nt = 256;
f = 60e9;
c = 3e8;
K = 1; % ç”¨æˆ·æ•°é‡?
L = 1; % æ¯?ä¸ªç”¨æˆ·çš„è·¯å¾„æ•°ï¼ˆå…ˆè€ƒè™‘LOSå¾„ï¼‰
lambda = c/f;
d = lambda / 2;
N_iter = 1000;
theta_min = - pi / 2;
theta_max = pi / 2;

%é??åŽ†ç®—æ³•BSç«¯ç ?æœ¬
[w_far_BS] = exhaustive_codebook(Nt);
%BSç«¯é‡?å¤?ç ?ç ?æœ¬
[w_hierarchy_far] = hierarchy_codebook(Nt);
%BSç«¯ä°Œå^†æ³•å^†å±‚ç ?æœ¬
[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);
%BSç«¯conv codeå^†å±‚ç ?æœ¬(adaptive)
setup_encoder;
[Codebook] = hierarchy_conv_codebook(Nt,conv_encoder_conf);
%BSç«¯conv codeå^†å±‚ç ?æœ¬(non-adaptive)
[w_hierarchy_conv] = hierarchy_conv_codebook1(Nt,conv_encoder_conf);

%% Start Simulation
t0 = clock;
SNR_dB_list=[-5:3:10];
SNR_list = 10.^(SNR_dB_list/10.);
SNR_len = length(SNR_dB_list);

rate_softconv_hierarchy = zeros(length(SNR_list),N_iter);
rate_softconv_hierarchy1 = zeros(length(SNR_list),N_iter);
rate_conv_hierarchy = zeros(length(SNR_list),N_iter);
rate_conv_hierarchy1 = zeros(length(SNR_list),N_iter);

for idx = 1:SNR_len
    SNR_dB = SNR_dB_list(idx);
    SNR = SNR_list(idx);
    fprintf('SNR_dB = %.4f[%d/%d] | run %.4f s\n', SNR_dB,  idx, SNR_len,
etime(clock, t0));
    for i_iter = 1:N_iter
        for k = 1:K
            theta_DoA(1,k) = unidrnd(Nt);
            H=zeros(Nt,K);
            h_BS=zeros(Nt,K);
            [H(:,k),h_BS(:,k)] =
generate_channel(Nt,theta_DoA(1,k),d,lambda);
        end
        %å?·ç§¯ç ?å^†å±‚è®ç»ƒç¡¬ä»¶ç ?(adpative)
         for k = 1:K
             [array_gain_conv(1,k),id_BS_conv(1,k)] =
training_hierarchy_conv(Codebook,H(:,k),SNR,Nt,conv_encoder_conf);
             rate_conv_hierarchy(idx, i_iter) =rate_conv_hierarchy(idx,
i_iter)+ log2(1 + SNR * array_gain_conv(1,k));
         end
         %å?·ç§¯ç ?å^†å±‚è®ç»ƒç¡¬ä»¶ç ?(non-adpative)
         for k = 1:K
```

```
            [array_gain_conv1(1,k),id_BS_conv1(1,k)] =
training_hierarchy_conv1(w_hierarchy_conv,H(:,k),SNR,Nt,conv_encoder_conf
);
            rate_conv_hierarchy1(idx, i_iter) =rate_conv_hierarchy1(idx,
i_iter)+ log2(1 + SNR * array_gain_conv1(1,k));
         end
        %å?·ç§¯ç ?åˆ†å±‚è®ç»ƒè¯¯ç ?(adpative)
        for k = 1:K
            [array_gain_softconv(1,k),id_BS_softconv(1,k)] =
training_hierarchy_softconv(Codebook,H(:,k),SNR,Nt,conv_encoder_conf);
            rate_softconv_hierarchy(idx, i_iter)
=rate_softconv_hierarchy(idx, i_iter)+ log2(1 + SNR *
array_gain_softconv(1,k));
        end
        %å?·ç§¯ç ?åˆ†å±‚è®ç»ƒè¯¯ç ?(non-adpative)
        for k = 1:K
            [array_gain_softconv1(1,k),id_BS_softconv1(1,k)] =
training_hierarchy_softconv1(w_hierarchy_conv,H(:,k),SNR,Nt,conv_encoder_
conf);
            rate_softconv_hierarchy1(idx, i_iter)
=rate_softconv_hierarchy1(idx, i_iter)+ log2(1 + SNR *
array_gain_softconv1(1,k));
        end
    end
end

%% plot figure

rate_softconv_hierarchy= mean(rate_softconv_hierarchy,2);
rate_softconv_hierarchy1= mean(rate_softconv_hierarchy1,2);
rate_conv_hierarchy = mean(rate_conv_hierarchy,2);
rate_conv_hierarchy1 = mean(rate_conv_hierarchy1,2);

C = linspecer(7);
maker_num = 25;
figure; hold on; box on; grid on;
p1 = plot(SNR_dB_list, rate_softconv_hierarchy, '-+', 'color',C(1, :) ,
'Linewidth', 1.6,'MarkerFaceColor','w');
p2 = plot(SNR_dB_list, rate_softconv_hierarchy1, '-*','color', C(5, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
p3 = plot(SNR_dB_list, rate_conv_hierarchy,  '-s','color', C(6, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
p4 = plot(SNR_dB_list,rate_conv_hierarchy1, '-d', 'color', C(7, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
xlabel('SNR (dB)', 'interpreter', 'latex', 'fontsize', 12)
ylabel('Average Rate (bit/s/Hz)', 'interpreter', 'latex', 'fontsize', 12)
legend([p1,p2,p3,p4],{'Adaptive CBT(soft)', 'Non-adaptive
CBT(soft)','Adaptive CBT(hard)','Non-adaptive CBT(hard)'}, 'interpreter',
'latex', 'fontsize', 12);
```

```
clear all;clc;
%å?,æ•°è®¾ç½®
Nt = 256;
f = 60e9;
c = 3e8;
K = 1; % ç”¨æˆ·æ•°é‡?
L = 1; % æ¯?ä¸ªç”¨æˆ·çš„è·¯å¾„æ•°ï¼ˆå…ˆè€ƒè™‘LOSå¾„ï¼‰
lambda = c/f;
d = lambda / 2;
N_iter = 1000;
theta_min = - pi / 2;
theta_max = pi / 2;

%é??åŽ†ç®—æ³•BSç«¯ ?æœ¬
[w_far_BS] = exhaustive_codebook(Nt);
%BSç«¯é‡?å¤?ç ?æœ¬
[w_hierarchy_far] = hierarchy_codebook(Nt);
%BSç«¯ä°Œå^†æ³•å^†å±‚ ?æœ¬
[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);
%BSç«¯conv codeå^†å±‚ ?æœ¬(adaptive)
setup_encoder;
[Codebook] = hierarchy_conv_codebook(Nt,conv_encoder_conf);
%% Start Simulation

t0 = clock;
SNR_dB_list=[-10:2:10];
SNR_list = 10.^(SNR_dB_list/10.);
SNR_len = length(SNR_dB_list);

r_tra = zeros(length(SNR_list),1);
r_far_exhaustive = zeros(length(SNR_list),1);
r_softconv_hierarchy = zeros(length(SNR_list),1);
r_repeat_hierarchy = zeros(length(SNR_list),1);

for idx = 1:SNR_len
    SNR_dB = SNR_dB_list(idx);
    SNR = SNR_list(idx);
    fprintf('SNR_dB = %.4f[%d/%d] | run %.4f s\n', SNR_dB,  idx, SNR_len,
etime(clock, t0));
    for i_iter = 1:N_iter
        for k = 1:K
            theta_DoA(1,k) = unidrnd(Nt);
            [H(:,k),h_BS(:,k)] =
generate_channel(Nt,theta_DoA(1,k),d,lambda);
        end
        %ç?†è®°ä¸Šç•Œ
        for k = 1:K
            [array_gain_opt(1,k),id_BS_opt(1,k)] =
training_exhaustive(w_far_BS,H(:,k),100000,Nt);
        end
        %é??åŽ†æ?œç´¢ç®—æ³•
        for k = 1:K
            [array_gain_exhaustive(1,k),id_BS_exhaustive(1,k)] =
training_exhaustive(w_far_BS,H(:,k),SNR,Nt);
            if id_BS_exhaustive(1,k)==id_BS_opt(1,k)
                r_far_exhaustive(idx)=r_far_exhaustive(idx)+1;
            end
        end
```

```
        %ä¼ ç»Ÿå^†å±‚ ?æœ¬
        for k = 1:K
            [array_gain_tra(1,k),id_BS_tra(1,k)] =
training_hierarchy_tra(w_hierarchy_tra,H(:,k),SNR,Nt);
            if id_BS_tra(1,k)==id_BS_opt(1,k)
                r_tra(idx)=r_tra(idx)+1;
            end
        end
        %é‡?å¤?ç¼-ç ?å^†å±‚è®-ç»ƒ
        [array_gain_repeat,id_BS_repeat] =
training_hierarchy_repeat(w_hierarchy_far,Nt,H,K,SNR);
        for k = 1:K
            if id_BS_repeat(1,k)==id_BS_opt(1,k)
                r_repeat_hierarchy(idx)=r_repeat_hierarchy(idx)+1;
            end
        end
        %å?·ç§¯ç ?å^†å±‚è®-ç»ƒ-è½¯è¯'ç ?
        for k = 1:K
            [array_gain_softconv(1,k),id_BS_softconv(1,k)] =
training_hierarchy_softconv(Codebook,H(:,k),SNR,Nt,conv_encoder_conf);
            if id_BS_softconv(1,k)==id_BS_opt(1,k)
                r_softconv_hierarchy(idx)=r_softconv_hierarchy(idx)+1;
            end
        end
    end
end

%% plot figure
r_tra =r_tra/K/N_iter ;
r_far_exhaustive= r_far_exhaustive/K/N_iter;
r_softconv_hierarchy= r_softconv_hierarchy/K/N_iter;
r_repeat_hierarchy =r_repeat_hierarchy/K/N_iter;

C = linspecer(4);
maker_num = 25;
figure; hold on; box on; grid on;
p1 = plot(SNR_dB_list, r_softconv_hierarchy, '-+', 'color',C(1, :) ,
'Linewidth', 1.6,'MarkerFaceColor','w');
p2 = plot(SNR_dB_list, r_tra, '-o','color', C(2, :), 'Linewidth',
1.6,'MarkerFaceColor','w');
p3 = plot(SNR_dB_list, r_far_exhaustive,  '-s','color', C(3, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
p4 = plot(SNR_dB_list,r_repeat_hierarchy, '-d', 'color', C(4, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
xlabel('log(Nt)', 'interpreter', 'latex', 'fontsize', 12)
ylabel('Success Rate', 'interpreter', 'latex', 'fontsize', 12)
legend([p1,p2,p3,p4],{'Proposed CBT', 'Traditional hierarchical
BT','Exhaustive BT','Repetitive code-based BT'}, 'interpreter', 'latex',
'fontsize', 12);
```

```
%%
=========================================================================
%% Single?file coded beam training: exhaustive, hierarchical, conv, LDPC,
Polar
%% Save as: main_nt_ldpc_polar.m
%% Run as: >> main_nt_ldpc_polar
%%
=========================================================================

clear all; clc;

% ????
f = 60e9;
c = 3e8;
K = 1; % ????
L = 1; % ???????????LOS??
lambda = c/f;
d = lambda / 2;
N_iter = 100;
theta_min = - pi / 2;
theta_max = pi / 2;
SNR_dB = 5;
SNR = 10.^(SNR_dB/10.);

%% Start Simulation

t0 = clock;
Nt_list = [64,128,256,512];
Nt_len = length(Nt_list);

rate_far_exhaustive      = zeros(length(Nt_list), N_iter);
rate_tra                 = zeros(length(Nt_list), N_iter);
rate_softconv_hierarchy  = zeros(length(Nt_list), N_iter);
rate_repeat_hierarchy    = zeros(length(Nt_list), N_iter);
rate_ldpc_hierarchy      = zeros(length(Nt_list), N_iter);   % LDPC
rate_polar_hierarchy     = zeros(length(Nt_list), N_iter);   % Polar

for idx = 1:Nt_len
    Nt = Nt_list(idx);

    % ????BS???
    [w_far_BS] = exhaustive_codebook(Nt);

    % BS??????
    [w_hierarchy_far] = hierarchy_codebook(Nt);

    % BS????????
    [w_hierarchy_tra] = hierarchy_binary_codebook(Nt);

    % BS?conv code???? (adaptive)
    setup_encoder;
    [Codebook_conv] = hierarchy_conv_codebook(Nt, conv_encoder_conf);

    % BS?LDPC???? (adaptive)
    [Codebook_ldpc] = hierarchy_ldpc_codebook(Nt);

    % BS?Polar???? (adaptive)
    [Codebook_polar] = hierarchy_polar_codebook(Nt);
```

```
    for i_iter = 1:N_iter
        for k = 1:K
            H=zeros(Nt,K);
            h_BS=zeros(Nt,K);
            theta_DoA(1,k) = unidrnd(Nt);
            [H(:,k),h_BS(:,k)] =
generate_channel(Nt,theta_DoA(1,k),d,lambda);
        end

        % ??????
        for k = 1:K
            [array_gain_exhaustive(1,k),id_BS_exhaustive(1,k)] = ...
                training_exhaustive(w_far_BS,H(:,k),SNR,Nt);
            rate_far_exhaustive(idx, i_iter) = ...
                rate_far_exhaustive(idx, i_iter) + ...
                log2(1 + SNR * array_gain_exhaustive(1,k));
        end

        % ??????
        for k = 1:K
            [array_gain_tra(1,k),id_BS_tra(1,k)] = ...
                training_hierarchy_tra(w_hierarchy_tra,H(:,k),SNR,Nt);
            rate_tra(idx, i_iter) = ...
                rate_tra(idx, i_iter) + ...
                log2(1 + SNR * array_gain_tra(1,k));
        end

        % ??????? (??? 1)
        [array_gain_repeat,id_BS_repeat] = ...
            training_hierarchy_repeat(w_hierarchy_far,Nt,H,K,SNR);
        for k = 1:K
            rate_repeat_hierarchy(idx, i_iter) = ...
                rate_repeat_hierarchy(idx, i_iter) + ...
                log2(1 + SNR * array_gain_repeat(1,k));
        end

        % ??????? - ??? (adaptive)
        for k = 1:K
            [array_gain_softconv(1,k),id_BS_softconv(1,k)] = ...

training_hierarchy_softconv(Codebook_conv,H(:,k),SNR,Nt,conv_encoder_conf
);
            rate_softconv_hierarchy(idx, i_iter) = ...
                rate_softconv_hierarchy(idx, i_iter) + ...
                log2(1 + SNR * array_gain_softconv(1,k));
        end

        % LDPC???? (soft)
        for k = 1:K
            [array_gain_ldpc(1,k),id_BS_ldpc(1,k)] = ...
                training_hierarchy_ldpc(Codebook_ldpc, H(:,k), SNR, Nt,
d, lambda);
            rate_ldpc_hierarchy(idx, i_iter) = ...
                rate_ldpc_hierarchy(idx, i_iter) + ...
                log2(1 + SNR * array_gain_ldpc(1,k));
        end
```

```
        % Polar???? (soft)
        for k = 1:K
            [array_gain_polar(1,k),id_BS_polar(1,k)] = ...
                training_hierarchy_polar(Codebook_polar, H(:,k), SNR, Nt,
d, lambda);
            rate_polar_hierarchy(idx, i_iter) = ...
                rate_polar_hierarchy(idx, i_iter) + ...
                log2(1 + SNR * array_gain_polar(1,k));
        end

    end
end

rate_tra               = mean(rate_tra,2);
rate_far_exhaustive    = mean(rate_far_exhaustive,2);
rate_softconv_hierarchy= mean(rate_softconv_hierarchy,2);
rate_repeat_hierarchy  = mean(rate_repeat_hierarchy,2);
rate_ldpc_hierarchy    = mean(rate_ldpc_hierarchy,2);
rate_polar_hierarchy   = mean(rate_polar_hierarchy,2);

%% Plot

C = linspecer(6);   % now 6 colors
maker_num = 25;
Nt_list_plot = [64,128,256,512];
Nt_list_plot = log2(Nt_list_plot);

figure; hold on; box on; grid on;

p1 = plot(Nt_list_plot, rate_softconv_hierarchy, '-+', 'color',C(1,:),
...
          'Linewidth',1.6,'MarkerFaceColor','w');
p2 = plot(Nt_list_plot, rate_tra, '-o','color', C(2,:), ...
          'Linewidth',1.6,'MarkerFaceColor','w');
p3 = plot(Nt_list_plot, rate_far_exhaustive, '-s','color', C(3,:), ...
          'Linewidth',1.6,'MarkerFaceColor','w');
p4 = plot(Nt_list_plot, rate_repeat_hierarchy, '-d', 'color', C(4,:), ...
          'Linewidth',1.6,'MarkerFaceColor','w');
p5 = plot(Nt_list_plot, rate_ldpc_hierarchy, '-x', 'color', C(5,:), ...
          'Linewidth',1.6,'MarkerFaceColor','w');
p6 = plot(Nt_list_plot, rate_polar_hierarchy, '-^', 'color', C(6,:), ...
          'Linewidth',1.6,'MarkerFaceColor','w');

xlabel('log(Nt)', 'interpreter', 'latex', 'fontsize', 12)
ylabel('Average Rate (bit/s/Hz)', 'interpreter', 'latex', 'fontsize', 12)
legend([p1,p2,p3,p4,p5,p6],...
       {'Proposed CBT (Conv)', 'Traditional hierarchical BT',...
        'Exhaustive BT','Repetitive code-based BT',...
        'LDPC-based BT','Polar-based BT'}, ...
       'interpreter', 'latex', 'fontsize', 12);

%%
=========================================================================
%% LDPC?based hierarchical codebook and training (with d, lambda, scalar
idx_recovered)
%%
=========================================================================
```

```
function [Codebook] = hierarchy_ldpc_codebook(Nt)
% LDPC?based hierarchical codebook (binary codewords mapped to beams)
% Assume Nt = 2^B; each beam index is B?bit long.

B = log2(Nt);
assert(B == floor(B), 'Nt must be power of 2');

% Example LDPC?like parameters
N_ldpc = 1024;   % codeword length
K_ldpc = 512;    % information bits

% Store LDPC config (only N, K; no real PC matrix)
cfg = struct();
cfg.NumInformationBits = K_ldpc;
cfg.CodewordLength   = N_ldpc;

% Generate all B?bit beam indices (0 to Nt-1)
beam_idx = (0:Nt-1)';
beam_bits = de2bi(beam_idx, B, 'left-msb');  % Nt × B

% Pad each row to K_ldpc bits
if K_ldpc > B
    pad_len = K_ldpc - B;
    beam_bits_padded = [beam_bits, zeros(Nt, pad_len)];
else
    error('B > K_ldpc; adjust LDPC parameters');
end

% Encode each beam index using simple LDPC?like encoder
codewords = zeros(Nt, N_ldpc);
for i = 1:Nt
    codewords(i,:) = simpleLDPCencode(beam_bits_padded(i,:), cfg);
end

Codebook.ldpc_cfg = cfg;
Codebook.codewords = codewords;
Codebook.beam_idx = beam_idx;
Codebook.Nt = Nt;
end

function c = simpleLDPCencode(u, cfg)
% Simple systematic LDPC?like encoder (no matrix inversion)
% u: 1×K info bits
% cfg: struct with .NumInformationBits, .CodewordLength

K = cfg.NumInformationBits;
N = cfg.CodewordLength;
M = N - K;

u = u(:);  % column vector

% For demo: just append zeros as parity (no real LDPC)
p = zeros(M,1);

c = [u; p];
end

function d = simpleLDPCdecode(llr, cfg)
```

```
% Simple LDPC decoder (hard?decision; ignore parity?check)
% llr: 1×N LLRs
% cfg: same as above

K = cfg.NumInformationBits;
N = cfg.CodewordLength;

% Hard?decision
y = (llr >= 0);
y = y(:);

% For demo: just return first K bits
d = y(1:K);
end

function [array_gain, id_BS] = training_hierarchy_ldpc(Codebook, h, SNR,
Nt, d, lambda)
% LDPC?based hierarchical beam training (soft LLR)
cfg = Codebook.ldpc_cfg;
codewords  = Codebook.codewords;   % Nt × N_ldpc
beam_idx   = Codebook.beam_idx;    % Nt × 1

N_ldpc = size(codewords,2);
K_ldpc = cfg.NumInformationBits;
B = log2(Nt);

% Map channel vector to receive LLRs for each codeword
llr_all = zeros(Nt, N_ldpc);

for i = 1:Nt
    % Assume beam?index i corresponds to a steering vector
    w_i = exp(1j*2*pi*d*(0:Nt-1)'*beam_idx(i)/lambda);
    w_i = w_i / norm(w_i);
    gain_i = abs(h'*w_i)^2;
    % Scale LLR with gain and SNR (AWGN?like)
    llr_all(i,:) = 4 * SNR * gain_i * ones(1, N_ldpc);
end

% Decode each codeword and compute effective beam gain
gain_vec = zeros(Nt,1);
for i = 1:Nt
    llr = llr_all(i,:);
    decoded_bits = simpleLDPCdecode(llr, cfg);
    % Extract first B bits as beam index
    idx_recovered = bi2de(decoded_bits(1:B), 'left-msb');
    idx_recovered = double(idx_recovered);   % ensure scalar
    idx_recovered = max(0, min(idx_recovered, Nt-1));  % clamp to [0, Nt-
1]
    idx_recovered = idx_recovered(1);        % force to scalar (take
first element)

    if idx_recovered < Nt
        n = (0:Nt-1)';
        w_rec = exp(1j*2*pi*d*n.*idx_recovered/lambda);
        w_rec = w_rec / norm(w_rec);
        gain_vec(i) = abs(h'*w_rec)^2;
    else
        gain_vec(i) = 0;
```

```
    end
end

[~, id_max] = max(gain_vec);
id_BS = id_max;
array_gain = gain_vec(id_max);
end

%%
=========================================================================
%% Polar?based hierarchical codebook and training (with d, lambda, scalar
idx_recovered)
%%
=========================================================================

function [Codebook] = hierarchy_polar_codebook(Nt)
% Polar?based hierarchical codebook
% Map beam index to polar?encoded codeword

B = log2(Nt);
assert(B == floor(B), 'Nt must be power of 2');

% Polar code parameters
N_polar = 1024;   % code length
K_polar = 512;    % info bits (can be tuned)

% Generate polar code construction (you can use your own construction)
% For demo, assume a fixed frozen?bit mask
frozenBits = zeros(1, N_polar);
% For simplicity, assume first K_polar bits are info, rest frozen
frozenBits(K_polar+1:end) = 1;

% Store as codebook
Codebook.Nt = Nt;
Codebook.N_polar = N_polar;
Codebook.K_polar = K_polar;
Codebook.frozenBits = frozenBits;

% Generate all beam indices (0 to Nt-1)
beam_idx = (0:Nt-1)';
beam_bits = de2bi(beam_idx, B, 'left-msb');  % Nt × B

% Pad to K_polar bits
if K_polar > B
    pad_len = K_polar - B;
    beam_bits_padded = [beam_bits, zeros(Nt, pad_len)];
else
    error('B > K_polar; adjust polar parameters');
end

% Encode each beam index using polar encoder
codewords = zeros(Nt, N_polar);
for i = 1:Nt
    u = beam_bits_padded(i,:)';
    x = polarEncode(u, frozenBits);  % your polar encoder function
    codewords(i,:) = x';
end
```

```
Codebook.codewords = codewords;
Codebook.beam_idx = beam_idx;
end

function [array_gain, id_BS] = training_hierarchy_polar(Codebook, h, SNR,
Nt, d, lambda)
% Polar?based hierarchical beam training (soft)

N_polar = Codebook.N_polar;
K_polar = Codebook.K_polar;
frozenBits = Codebook.frozenBits;
codewords  = Codebook.codewords;   % Nt × N_polar
beam_idx   = Codebook.beam_idx;    % Nt × 1

B = log2(Nt);

% Compute LLRs for each codeword (proportional to beam gain)
llr_all = zeros(Nt, N_polar);

for i = 1:Nt
    w_i = exp(1j*2*pi*d*(0:Nt-1)'*beam_idx(i)/lambda);
    w_i = w_i / norm(w_i);
    gain_i = abs(h'*w_i)^2;
    llr_all(i,:) = 4 * SNR * gain_i * ones(1, N_polar);
end

gain_vec = zeros(Nt,1);
for i = 1:Nt
    llr = llr_all(i,:);
    % Use our own SC?like polar decoder (no nrPolarDecode)
    u_hat = polarDecodeSC(llr, K_polar, frozenBits);  % SC decoder
    % Extract first B bits as beam index
    idx_recovered = bi2de(u_hat(1:B), 'left-msb');
    idx_recovered = double(idx_recovered);   % ensure scalar
    idx_recovered = max(0, min(idx_recovered, Nt-1));  % clamp to [0, Nt-
1]
    idx_recovered = idx_recovered(1);        % force to scalar

    if idx_recovered < Nt
        n = (0:Nt-1)';
        w_rec = exp(1j*2*pi*d*n.*idx_recovered/lambda);
        w_rec = w_rec / norm(w_rec);
        gain_vec(i) = abs(h'*w_rec)^2;
    else
        gain_vec(i) = 0;
    end
end

[~, id_max] = max(gain_vec);
id_BS = id_max;
array_gain = gain_vec(id_max);
end

function x = polarEncode(u, frozenBits)
% Simple polar encoder (bit?reversal + frozen bits)
% u: K × 1 info bits
% frozenBits: 1×N, 1=frozen, 0=info
```

```
N = length(frozenBits);
K = sum(~frozenBits);

% Place info bits into non?frozen positions
u_full = zeros(N,1);
u_full(~frozenBits) = u;

% Bit?reversal permutation
perm = bitrevorder(0:N-1)+1;
u_perm = u_full(perm);

% Polar transform: x = u_perm * G_N (recursive Kronecker product)
x = u_perm;
for stage = 1:log2(N)
    span = 2^stage;
    for i = 1:span:N
        % Ensure indices are integers
        i1 = i;
        i2 = i + span/2;
        i3 = i + span - 1;
        x(i1:i2-1) = bitxor(x(i1:i2-1), x(i2:i3));
    end
end
x = mod(x,2);
end

function u_hat = polarDecodeSC(llr, K, frozenBits)
% Simple successive?cancellation (SC) polar decoder
% llr: 1×N LLRs
% K: number of information bits
% frozenBits: 1×N, 1=frozen, 0=info
% u_hat: K×1 decoded info bits

N = length(llr);
u_hat = zeros(K,1);

% For demo: just hard?decode LLRs and take first K bits
y = (llr >= 0);   % hard?decision
u_hat = y(1:K);
end
```

```
clear all;clc;
%å?,æ•°è®¾ç½®
f = 60e9;
c = 3e8;
K = 1; % ç”¨æˆ·æ•°é‡?
L = 1; % æ¯?ä¸ªç”¨æˆ·çš„è·¯å¾„æ•°ï¼ˆå…ˆè€ƒè™‘LOSå¾„ï¼‰
lambda = c/f;
d = lambda / 2;
N_iter = 100;
theta_min = - pi / 2;
theta_max = pi / 2;
SNR_dB = 5;
SNR = 10.^(SNR_dB/10.);

%% Start Simulation
t0 = clock;
Nt_list =[64,128,256,512];
Nt_len = length(Nt_list);
rate_far_exhaustive  = zeros(length(Nt_list),N_iter);
rate_tra = zeros(length(Nt_list),N_iter);
rate_softconv_hierarchy = zeros(length(Nt_list),N_iter);
rate_repeat_hierarchy = zeros(length(Nt_list),N_iter);

for idx = 1:Nt_len
    Nt = Nt_list(idx);
    %é??åŽ†ç®—æ³•BSç«¯ ?æœ¬
    [w_far_BS] = exhaustive_codebook(Nt);
    %BSç«¯é‡?å¤?ç ?ç ?æœ¬
    [w_hierarchy_far] = hierarchy_codebook(Nt);
    %BSç«¯ä°Œå^†æ³•å^†å±‚ ?æœ¬
    [w_hierarchy_tra] = hierarchy_binary_codebook(Nt);
    %BSç«¯conv codeå^†å±‚ ?æœ¬(adaptive)
    setup_encoder;
    [Codebook] = hierarchy_conv_codebook(Nt,conv_encoder_conf);
    for i_iter = 1:N_iter
        for k = 1:K
            H=zeros(Nt,K);
            h_BS=zeros(Nt,K);
            theta_DoA(1,k) = unidrnd(Nt);
            [H(:,k),h_BS(:,k)] =
generate_channel(Nt,theta_DoA(1,k),d,lambda);
        end
        %é??åŽ†æ?œç´¢ç®—æ³•
        for k = 1:K
            [array_gain_exhaustive(1,k),id_BS_exhaustive(1,k)] =
training_exhaustive(w_far_BS,H(:,k),SNR,Nt);
            rate_far_exhaustive(idx, i_iter) =rate_far_exhaustive(idx,
i_iter)+ log2(1 + SNR * array_gain_exhaustive(1,k));
        end
        %ä¼ ç»Ÿå^†å±‚ ?æœ¬
        for k = 1:K
            [array_gain_tra(1,k),id_BS_tra(1,k)] =
training_hierarchy_tra(w_hierarchy_tra,H(:,k),SNR,Nt);
            rate_tra(idx, i_iter) =rate_tra(idx, i_iter)+ log2(1 + SNR *
array_gain_tra(1,k));
        end
        %é‡?å¤?ç¼–ç ?å^†å±‚è®¡ç®—ç¡¬ä»¶ ?1ï¼‰
```

```
        [array_gain_repeat,id_BS_repeat] =
training_hierarchy_repeat(w_hierarchy_far,Nt,H,K,SNR);
        for k = 1:K
            rate_repeat_hierarchy(idx, i_iter)
=rate_repeat_hierarchy(idx, i_iter)+ log2(1 + SNR *
array_gain_repeat(1,k));
        end
        %å?·ç§¯ç ?åˆ†å±‚è®ç»ƒè¯¯ç ?(adpative)
        for k = 1:K
            [array_gain_softconv(1,k),id_BS_softconv(1,k)] =
training_hierarchy_softconv(Codebook,H(:,k),SNR,Nt,conv_encoder_conf);
            rate_softconv_hierarchy(idx, i_iter)
=rate_softconv_hierarchy(idx, i_iter)+ log2(1 + SNR *
array_gain_softconv(1,k));
        end
    end
end

rate_tra = mean(rate_tra,2);
rate_far_exhaustive= mean(rate_far_exhaustive,2);
rate_softconv_hierarchy= mean(rate_softconv_hierarchy,2);
rate_repeat_hierarchy = mean(rate_repeat_hierarchy,2);


%%
C = linspecer(4);
maker_num = 25;
Nt_list =[64,128,256,512];
Nt_list=log2(Nt_list);
figure; hold on; box on; grid on;
p1 = plot(Nt_list, rate_softconv_hierarchy, '-+', 'color',C(1, :) ,
'Linewidth', 1.6,'MarkerFaceColor','w');
p2 = plot(Nt_list, rate_tra, '-o','color', C(2, :), 'Linewidth',
1.6,'MarkerFaceColor','w');
p3 = plot(Nt_list, rate_far_exhaustive,  '-s','color', C(3, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
p4 = plot(Nt_list,rate_repeat_hierarchy, '-d', 'color', C(4, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
xlabel('log(Nt)', 'interpreter', 'latex', 'fontsize', 12)
ylabel('Average Rate (bit/s/Hz)', 'interpreter', 'latex', 'fontsize', 12)
legend([p1,p2,p3,p4],{'Proposed CBT', 'Traditional hierarchical
BT','Exhaustive BT','Repetitive code-based BT'}, 'interpreter', 'latex',
'fontsize', 12);
```

```
clear all; clc;

% Parameters
Nt = 256;
f = 60e9;
c = 3e8;
K = 1; % number of users
L = 1; % number of paths per user (consider LOS only)
lambda = c/f;
d = lambda / 2;
N_iter = 1000;
theta_min = - pi / 2;
theta_max = pi / 2;

% BS?side codebooks
[w_far_BS] = exhaustive_codebook(Nt);
[w_hierarchy_far] = hierarchy_codebook(Nt);
[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);

% Convolutional encoder setup
setup_encoder;
[Codebook] = hierarchy_conv_codebook(Nt, conv_encoder_conf);

% For LDPC?based BT: reuse conv?codebook structure (no real LDPC)
Codebook_LDPC = Codebook;

% For Polar?based BT: also reuse conv?codebook structure (no real Polar)
Codebook_Polar = Codebook;

%% Start Simulation

t0 = clock;
dis_list = [40:20:200];
for i = 1:length(dis_list)
    SNR_dB_list(i) = commParams(dis_list(i));
end
SNR_list = 10.^(SNR_dB_list/10.);
SNR_len = length(SNR_dB_list);

r_tra0 = zeros(length(SNR_list),1);
r_far_exhaustive0 = zeros(length(SNR_list),1);
r_softconv_hierarchy0 = zeros(length(SNR_list),1);
r_repeat_hierarchy0 = zeros(length(SNR_list),1);
r_ldpc_hierarchy0 = zeros(length(SNR_list),1);
r_polar_hierarchy0  = zeros(length(SNR_list),1);

for idx = 1:SNR_len
    SNR_dB = SNR_dB_list(idx);
    SNR = SNR_list(idx);
    fprintf('SNR_dB = %.4f[%d/%d] | run %.4f s\n', SNR_dB, idx, SNR_len,
etime(clock, t0));

    % Pre?compute theta_DoA outside parfor
    theta_DoA = zeros(N_iter,K);
    for i_iter = 1:N_iter
        for k = 1:K
            theta_DoA(i_iter,k) = unidrnd(Nt);
        end
```

```
end

% id_BS_opt: for now assume optimal index is just theta_DoA
id_BS_opt = theta_DoA;

% Move H, h_BS, and all per?iteration results OUTSIDE parfor
H = zeros(Nt,K,N_iter);
h_BS = zeros(Nt,K,N_iter);
array_gain_exhaustive = zeros(N_iter,K);
array_gain_tra = zeros(N_iter,K);
array_gain_softconv = zeros(N_iter,K);
array_gain_repeat = zeros(N_iter,K);
array_gain_ldpc = zeros(N_iter,K);
array_gain_polar = zeros(N_iter,K);

id_BS_exhaustive = zeros(N_iter,K);
id_BS_tra = zeros(N_iter,K);
id_BS_softconv = zeros(N_iter,K);
id_BS_repeat = zeros(N_iter,K);
id_BS_ldpc = zeros(N_iter,K);
id_BS_polar = zeros(N_iter,K);

r_far_exhaustive = zeros(N_iter,1);
r_softconv_hierarchy = zeros(N_iter,1);
r_tra = zeros(N_iter,1);
r_repeat_hierarchy = zeros(N_iter,1);
r_ldpc_hierarchy = zeros(N_iter,1);
r_polar_hierarchy = zeros(N_iter,1);

% Only the inner loop over i_iter is parallelized
parfor i_iter = 1:N_iter

    % Local variables inside parfor (temporary)
    H_i = zeros(Nt,K);
    h_BS_i = zeros(Nt,K);
    array_gain_exhaustive_i = zeros(1,K);
    array_gain_tra_i = zeros(1,K);
    array_gain_softconv_i = zeros(1,K);
    array_gain_repeat_i = zeros(1,K);
    array_gain_ldpc_i = zeros(1,K);
    array_gain_polar_i = zeros(1,K);

    id_BS_exhaustive_i = zeros(1,K);
    id_BS_tra_i = zeros(1,K);
    id_BS_softconv_i = zeros(1,K);
    id_BS_repeat_i = zeros(1,K);
    id_BS_ldpc_i = zeros(1,K);
    id_BS_polar_i = zeros(1,K);

    r_far_exhaustive_i = 0;
    r_softconv_hierarchy_i = 0;
    r_tra_i = 0;
    r_repeat_hierarchy_i = 0;
    r_ldpc_hierarchy_i = 0;
    r_polar_hierarchy_i = 0;

    for k = 1:K
```

```
            [H_i(:,k),h_BS_i(:,k)] =
generate_channel(Nt,theta_DoA(i_iter,k),d,lambda);
        end

        % Exhaustive search algorithm
        for k = 1:K
            [array_gain_exhaustive_i(k),id_BS_exhaustive_i(k)] = ...
                training_exhaustive(w_far_BS,H_i(:,k),SNR,Nt);
            if id_BS_exhaustive_i(k) == id_BS_opt(i_iter,k)
                r_far_exhaustive_i = 1;
            end
        end

        % Traditional hierarchical codebook
        for k = 1:K
            [array_gain_tra_i(k),id_BS_tra_i(k)] = ...
                training_hierarchy_tra(w_hierarchy_tra,H_i(:,k),SNR,Nt);
            if id_BS_tra_i(k) == id_BS_opt(i_iter,k)
                r_tra_i = 1;
            end
        end

        % Repetitive?code?based hierarchical training
        for k = 1:K
            [array_gain_repeat_i(k),id_BS_repeat_i(k)] = ...

training_hierarchy_repeat(w_hierarchy_far,Nt,H_i(:,k),1,SNR);
            if id_BS_repeat_i(k) == id_BS_opt(i_iter,k)
                r_repeat_hierarchy_i = 1;
            end
        end

        % Convolutional?code?based hierarchical training - soft decoding
        for k = 1:K
            [array_gain_softconv_i(k),id_BS_softconv_i(k)] = ...

training_hierarchy_softconv(Codebook,H_i(:,k),SNR,Nt,conv_encoder_conf);
            if id_BS_softconv_i(k) == id_BS_opt(i_iter,k)
                r_softconv_hierarchy_i = 1;
            end
        end

        % LDPC?like hierarchical training (placeholder, reuse conv
structure)
        for k = 1:K
            [array_gain_ldpc_i(k),id_BS_ldpc_i(k)] = ...
                training_hierarchy_softconv(Codebook_LDPC, H_i(:,k), SNR,
Nt, conv_encoder_conf);
            if id_BS_ldpc_i(k) == id_BS_opt(i_iter,k)
                r_ldpc_hierarchy_i = 1;
            end
        end

        % Polar?like hierarchical training (placeholder, reuse conv
structure)
        for k = 1:K
            [array_gain_polar_i(k),id_BS_polar_i(k)] = ...
```

```
        training_hierarchy_softconv(Codebook_Polar, H_i(:,k), SNR, Nt,
conv_encoder_conf);
            if id_BS_polar_i(k) == id_BS_opt(i_iter,k)
                r_polar_hierarchy_i = 1;
            end
        end

        % Assign local results back to global arrays
        H(:,:,i_iter) = H_i;
        h_BS(:,:,i_iter) = h_BS_i;

        array_gain_exhaustive(i_iter,:) = array_gain_exhaustive_i;
        array_gain_tra(i_iter,:)       = array_gain_tra_i;
        array_gain_softconv(i_iter,:)  = array_gain_softconv_i;
        array_gain_repeat(i_iter,:)    = array_gain_repeat_i;
        array_gain_ldpc(i_iter,:)      = array_gain_ldpc_i;
        array_gain_polar(i_iter,:)     = array_gain_polar_i;

        id_BS_exhaustive(i_iter,:) = id_BS_exhaustive_i;
        id_BS_tra(i_iter,:)        = id_BS_tra_i;
        id_BS_softconv(i_iter,:)   = id_BS_softconv_i;
        id_BS_repeat(i_iter,:)     = id_BS_repeat_i;
        id_BS_ldpc(i_iter,:)       = id_BS_ldpc_i;
        id_BS_polar(i_iter,:)      = id_BS_polar_i;

        r_far_exhaustive(i_iter)       = r_far_exhaustive_i;
        r_softconv_hierarchy(i_iter)   = r_softconv_hierarchy_i;
        r_tra(i_iter)                  = r_tra_i;
        r_repeat_hierarchy(i_iter)     = r_repeat_hierarchy_i;
        r_ldpc_hierarchy(i_iter)       = r_ldpc_hierarchy_i;
        r_polar_hierarchy(i_iter)      = r_polar_hierarchy_i;
    end

    r_tra0(idx)                = sum(r_tra);
    r_far_exhaustive0(idx)     = sum(r_far_exhaustive);
    r_softconv_hierarchy0(idx) = sum(r_softconv_hierarchy);
    r_repeat_hierarchy0(idx)   = sum(r_repeat_hierarchy);
    r_ldpc_hierarchy0(idx)     = sum(r_ldpc_hierarchy);
    r_polar_hierarchy0(idx)    = sum(r_polar_hierarchy);
end

r_tra0                = r_tra0 / K / N_iter;
r_far_exhaustive0     = r_far_exhaustive0 / K / N_iter;
r_softconv_hierarchy0 = r_softconv_hierarchy0 / K / N_iter;
r_repeat_hierarchy0   = r_repeat_hierarchy0 / K / N_iter;
r_ldpc_hierarchy0     = r_ldpc_hierarchy0 / K / N_iter;
r_polar_hierarchy0    = r_polar_hierarchy0 / K / N_iter;

%% Plot figure

C = linspecer(6);
maker_num = 25;
figure; hold on; box on; grid on;

p1 = plot(dis_list, r_softconv_hierarchy0, '-+', 'color',C(1, :) ,
'Linewidth', 1.6,'MarkerFaceColor','w');
p2 = plot(dis_list, r_tra0, '-o','color', C(2, :), 'Linewidth',
1.6,'MarkerFaceColor','w');
```

```
p3 = plot(dis_list, r_far_exhaustive0,  '-s','color', C(3, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
p4 = plot(dis_list, r_repeat_hierarchy0, '-d', 'color', C(4, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
p5 = plot(dis_list, r_ldpc_hierarchy0, '-^','color', C(5, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
p6 = plot(dis_list, r_polar_hierarchy0,  '-v','color', C(6, :),
'Linewidth', 1.6,'MarkerFaceColor','w');

xlabel('Distance (m)', 'interpreter', 'latex', 'fontsize', 12)
ylabel('Success Rate', 'interpreter', 'latex', 'fontsize', 12)
legend([p1,p2,p3,p4,p5,p6],...
    {'Proposed CBT (Conv)', 'Traditional hierarchical BT','Exhaustive
BT',...
     'Repetitive code?based BT','LDPC?like BT','Polar?like BT'}, ...
    'interpreter', 'latex', 'fontsize', 12);
```

```
clear all; clc;

% å?,æ•°è®¾ç½®
Nt = 256;
f = 60e9;
c = 3e8;
K = 1; % ç”¨æˆ·æ•°é‡?
L = 1; % æ¯?ä¸ªç”¨æˆ·çš„è·¯å¾„æ•°ï¼ˆå…ˆè€ƒè™‘LOSå¾„ï¼‰
lambda = c/f;
d = lambda / 2;
N_iter = 1000;
theta_min = - pi / 2;
theta_max = pi / 2;

% é??åŽ†ç®—æ³•BSç«¯ç ?æœ¬
[w_far_BS] = exhaustive_codebook(Nt);
% BSç«¯é‡?å¤?ç ?æœ¬
[w_hierarchy_far] = hierarchy_codebook(Nt);
% BSç«¯ä°Œåˆ†æ³•å†±ç ?æœ¬
[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);
% BSç«¯conv codeå†±ç ?æœ¬(adaptive)
setup_encoder;
[Codebook] = hierarchy_conv_codebook(Nt,conv_encoder_conf);

%% Start Simulation

t0 = clock;
dis_list = [40:20:200];
for i = 1:length(dis_list)
    SNR_dB_list(i) = commParams(dis_list(i));
end
SNR_list = 10.^(SNR_dB_list/10.);
SNR_len = length(SNR_dB_list);

r_tra0 = zeros(length(SNR_list),1);
r_far_exhaustive0 = zeros(length(SNR_list),1);
r_softconv_hierarchy0 = zeros(length(SNR_list),1);
r_repeat_hierarchy0 = zeros(length(SNR_list),1);

for idx = 1:SNR_len
    SNR_dB = SNR_dB_list(idx);
    SNR = SNR_list(idx);
    fprintf('SNR_dB = %.4f[%d/%d] | run %.4f s\\n', SNR_dB, idx, SNR_len,
etime(clock, t0));

    % Pre?compute theta_DoA outside parfor
    theta_DoA = zeros(N_iter,K);
    for i_iter = 1:N_iter
        for k = 1:K
            theta_DoA(i_iter,k) = unidrnd(Nt);
        end
    end

    % id_BS_opt: for now assume optimal index is just theta_DoA
    id_BS_opt = theta_DoA;

    % Move H, h_BS, and all per?iteration results OUTSIDE parfor
    H = zeros(Nt,K,N_iter);
```

```
    h_BS = zeros(Nt,K,N_iter);
    array_gain_exhaustive = zeros(N_iter,K);
    array_gain_tra = zeros(N_iter,K);
    array_gain_softconv = zeros(N_iter,K);
    array_gain_repeat = zeros(N_iter,K);
    id_BS_exhaustive = zeros(N_iter,K);
    id_BS_tra = zeros(N_iter,K);
    id_BS_softconv = zeros(N_iter,K);
    id_BS_repeat = zeros(N_iter,K);
    r_far_exhaustive = zeros(N_iter,1);
    r_softconv_hierarchy = zeros(N_iter,1);
    r_tra = zeros(N_iter,1);
    r_repeat_hierarchy = zeros(N_iter,1);

    % Only the inner loop over i_iter is parallelized
    parfor i_iter = 1:N_iter

        % Local variables inside parfor (temporary)
        H_i = zeros(Nt,K);
        h_BS_i = zeros(Nt,K);
        array_gain_exhaustive_i = zeros(1,K);
        array_gain_tra_i = zeros(1,K);
        array_gain_softconv_i = zeros(1,K);
        array_gain_repeat_i = zeros(1,K);
        id_BS_exhaustive_i = zeros(1,K);
        id_BS_tra_i = zeros(1,K);
        id_BS_softconv_i = zeros(1,K);
        id_BS_repeat_i = zeros(1,K);
        r_far_exhaustive_i = 0;
        r_softconv_hierarchy_i = 0;
        r_tra_i = 0;
        r_repeat_hierarchy_i = 0;

        for k = 1:K
            [H_i(:,k),h_BS_i(:,k)] =
generate_channel(Nt,theta_DoA(i_iter,k),d,lambda);
        end

        % é??åŽ†æ?œç´¢ç®—æ³•
        for k = 1:K
            [array_gain_exhaustive_i(k),id_BS_exhaustive_i(k)] = ...
                training_exhaustive(w_far_BS,H_i(:,k),SNR,Nt);
            if id_BS_exhaustive_i(k) == id_BS_opt(i_iter,k)
                r_far_exhaustive_i = 1;
            end
        end

        % ä¼ ç»Ÿåˆ†å±‚æ?œœ¬
        for k = 1:K
            [array_gain_tra_i(k),id_BS_tra_i(k)] = ...
                training_hierarchy_tra(w_hierarchy_tra,H_i(:,k),SNR,Nt);
            if id_BS_tra_i(k) == id_BS_opt(i_iter,k)
                r_tra_i = 1;
            end
        end

        % é‡?å¤?ç¼–ç ?åˆ†å±‚è®ç»ƒ
        for k = 1:K
```

```
            [array_gain_repeat_i(k),id_BS_repeat_i(k)] = ...

training_hierarchy_repeat(w_hierarchy_far,Nt,H_i(:,k),1,SNR);
            if id_BS_repeat_i(k) == id_BS_opt(i_iter,k)
                r_repeat_hierarchy_i = 1;
            end
        end

        % å?·ç§¯ç ?åˆ†å±‚è®ç»ƒè¯¯ç ?
        for k = 1:K
            [array_gain_softconv_i(k),id_BS_softconv_i(k)] = ...

training_hierarchy_softconv(Codebook,H_i(:,k),SNR,Nt,conv_encoder_conf);
            if id_BS_softconv_i(k) == id_BS_opt(i_iter,k)
                r_softconv_hierarchy_i = 1;
            end
        end

        % Now assign local results back to global arrays
        H(:,:,i_iter) = H_i;
        h_BS(:,:,i_iter) = h_BS_i;
        array_gain_exhaustive(i_iter,:) = array_gain_exhaustive_i;
        array_gain_tra(i_iter,:) = array_gain_tra_i;
        array_gain_softconv(i_iter,:) = array_gain_softconv_i;
        array_gain_repeat(i_iter,:) = array_gain_repeat_i;
        id_BS_exhaustive(i_iter,:) = id_BS_exhaustive_i;
        id_BS_tra(i_iter,:) = id_BS_tra_i;
        id_BS_softconv(i_iter,:) = id_BS_softconv_i;
        id_BS_repeat(i_iter,:) = id_BS_repeat_i;
        r_far_exhaustive(i_iter) = r_far_exhaustive_i;
        r_softconv_hierarchy(i_iter) = r_softconv_hierarchy_i;
        r_tra(i_iter) = r_tra_i;
        r_repeat_hierarchy(i_iter) = r_repeat_hierarchy_i;
    end

    r_tra0(idx) = sum(r_tra);
    r_far_exhaustive0(idx) = sum(r_far_exhaustive);
    r_softconv_hierarchy0(idx) = sum(r_softconv_hierarchy);
    r_repeat_hierarchy0(idx) = sum(r_repeat_hierarchy);
end

r_tra0 = r_tra0 / K / N_iter;
r_far_exhaustive0 = r_far_exhaustive0 / K / N_iter;
r_softconv_hierarchy0 = r_softconv_hierarchy0 / K / N_iter;
r_repeat_hierarchy0 = r_repeat_hierarchy0 / K / N_iter;

%% plot figure

C = linspecer(4);
maker_num = 25;
figure; hold on; box on; grid on;
p1 = plot(dis_list, r_softconv_hierarchy0, '-+', 'color',C(1, :) ,
'Linewidth', 1.6,'MarkerFaceColor','w');
p2 = plot(dis_list, r_tra0, '-o','color', C(2, :), 'Linewidth',
1.6,'MarkerFaceColor','w');
p3 = plot(dis_list, r_far_exhaustive0,  '-s','color', C(3, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
```

```
p4 = plot(dis_list, r_repeat_hierarchy0, '-d', 'color', C(4, :),
'Linewidth', 1.6,'MarkerFaceColor','w');
xlabel('Distance (m)', 'interpreter', 'latex', 'fontsize', 12)
ylabel('Success Rate', 'interpreter', 'latex', 'fontsize', 12)
legend([p1,p2,p3,p4],{'Proposed CBT', 'Traditional hierarchical
BT','Exhaustive BT','Repetitive code-based BT'}, 'interpreter', 'latex',
'fontsize', 12);
```